Assignment 3: Discrepancy Analysis

November 21, 2025

Website: https://softwarearchitecture.wixsite.com/eecs4314assignment

Group Name: SourceArchitects

Team Members Information

Name	Student ID	Email
Sheida Khodabakhshian Khansari	215534340	sheidakh@my.yorku.ca
Vrinda Ashwar	218624692	vrinda03@my.yorku.ca
Matthew Lee	218256743	mattl7@my.yorku.ca
Werner Bagu-Paiz	21727772	wernerbp@my.yorku.ca
William Moran	218928960	willvm24@my.yorku.ca
Stefewn Johnson	219411099	stef9@my.yorku.ca

1. Abstract **2**
2. Introduction **2**
3. Reflexion Analysis Process **2**
4. Top-Level Comparison **3**
4.1. Descriptions of top-level conceptual and concrete architecture 3
4.2. Comparison of conceptual and concrete architecture at high level (Reflexion analysis) & Modifications 4
4.3. Rationales behind Dependency Divergences in the Concrete Architecture 6
5. Subsystem-Level Detailed Analysis-Image Processing Subsystem **11**
5.1. Descriptions of the conceptual and the concrete architecture of imgproc 11
5.2. Comparison of conceptual and concrete architecture of imgproc 13
5.3. Rationales behind Dependency Divergences in the Concrete Architecture of Imgproc 14
6. Use Cases **15**
7. Lessons Learned **16**
8. Conclusion **16**

1. Abstract

This report compares the conceptual architecture of OpenCV with the concrete architecture extracted using tools such as Understand and LSEdit. A reflection analysis is performed to identify convergences, divergences, and absences between the two levels. The results show that while the conceptual model presents a clean layered structure, the actual implementation contains more cross-module dependencies and optional links. At the subsystem level, the imgproc module shows differences as well in the conceptual versus concrete architecture. The reasons behind these discrepancies are explained, including performance requirements, backward compatibility, and architectural evolution.

2. Introduction

One of the key tasks in software architecture analysis is to understand how the implemented architecture of a software system corresponds with the intended design of the system. This assignment is aimed at developing a reflexion analysis to compare the conceptual architecture designed in Assignment 1 and the concrete architecture reverse-engineered in Assignment 2 on the OpenCV system. The architectural discrepancies can be identified through this comparison, which can be convergences, divergences, and absences across subsystem boundaries. The reflexion analysis in this report is given at two levels. We start with the top-level architecture of OpenCV and compare the relationships and dependencies that were initially envisioned between some of the key subsystems (core, imgproc, highui, video, and imgcodecs,etc.) with the actual dependencies expressed by or obtained using some of the tools of the static analysis package, including LSEdit and Understand [1]. Such a high-level comparison aids understanding of the architecture's general patterns and mismatches at a larger scale, showing where the implemented system fails to meet the initially planned layered form. Second, we perform a more thorough, subsystem-based analysis by choosing a central and rather complicated component of OpenCV, which is the image processing subsystem (imgproc) [2]. In this, we examine internal second-level subsystems and dependency relationships, intending to determine how actual interactions compare with the conceptual expectations. This close analysis displays the small inconsistencies that are not seen on the upper level, which provide a more detailed insight into the architecture. Once the discrepancies have been found, the report will show rationales on why they happened, including architectural evolution, performance optimization, and optional module dependencies. Through these findings, we recommend mitigation measures to enhance architectural alignment, such as modifications to the concrete design. Lastly, the report will have supporting visual artifacts, such as updated sequence diagrams, as well as a discussion of lessons learned during the analysis process. Collectively, these elements of the report will provide an in-depth assessment of the correspondence between the theoretical design of OpenCV and the actual implementation, and how the reflexion analysis can inform future architectural development.

3. Reflexion Analysis Process

As seen in following section 4 of the report, the reflexion analysis process was used on the top-level architecture. This for example includes highgui, video, and core modules. The process was also used for the imgproc subsystem to catch finer relationships at the design level in section 5 of the report. We can see that every conceptual subsystem was mapped to a concrete subsystem. This was done by using visualization tools and static analysis. The analysis of the

extracted dependencies using Understand and their visualization using LSEdit, enabled us to have a consistent correspondence between OpenCV's code that was actually implemented and their designed modules described in OpenCV's documentation .

When the mapping part was settled, dependencies and interactions within the subsystems were the next part to analyze. There were three categories of relationships; divergent, absent, and convergent . Divergent is something that only shows up in the concrete subsystem only. Absent is something that was in the conceptual subsystem but not in the concrete. And finally, convergent dependencies are those that appear in both models . We modified our conceptual architecture diagram based on the dependencies extracted using Understand and visualized through LSEdit to show these relations]. This helped us discover exactly where the implementation diverged from the design intentions outlined in the OpenCV documentation.

Every discrepancy was checked thoroughly to find its rationale as described in the following sections. This was done through examining the visualized extracted dependencies using LSEdit and analyzing those dependencies through OpenCV's github repository with the help of DeepWiki [3]. This AI tool was used to help us quickly discover the github code commits that added specific dependencies. Some examples of rationals for differences between the conceptual and concrete architecture were modular refactoring, performance optimization, or architectural evolution when there's newer versions in OpenCV. These rationals are described in the following section.

4. Top-Level Comparison

4.1. Descriptions of top-level conceptual and concrete architecture

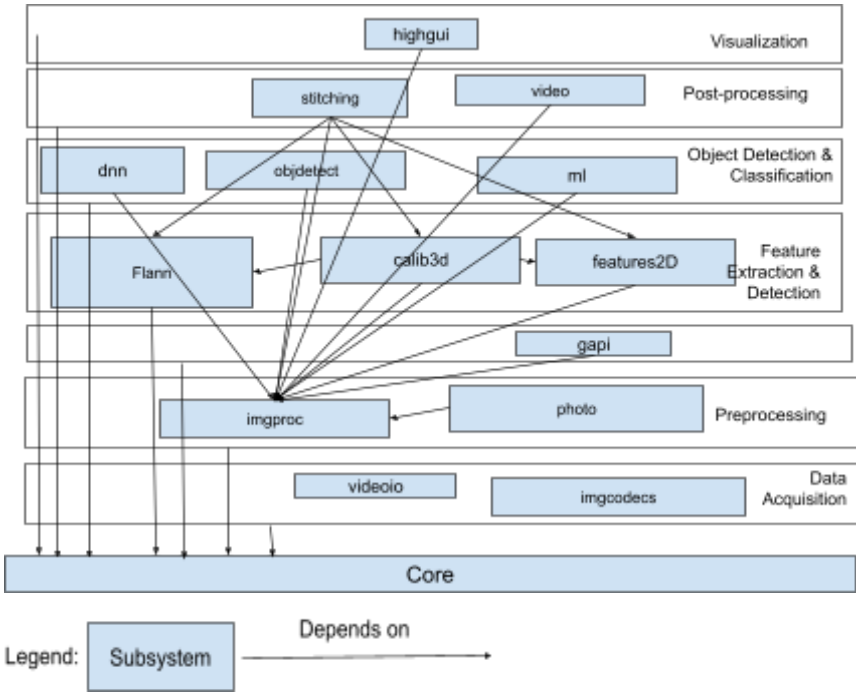

Figure 1. Conceptual Architecture

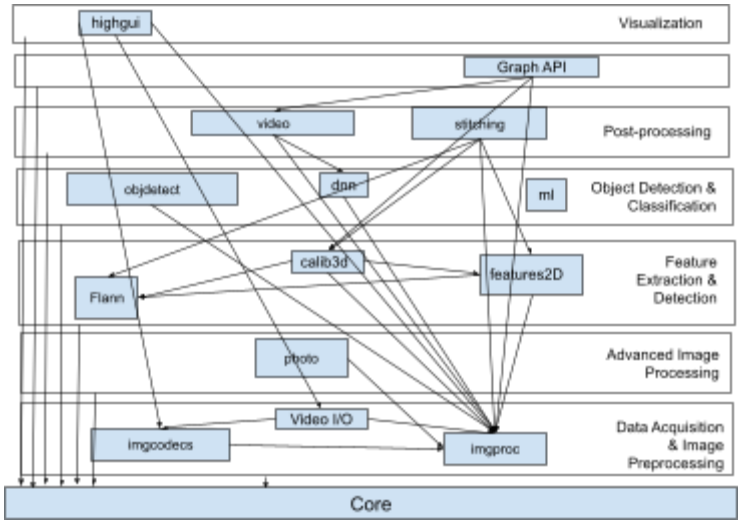

Figure 2. Derived concrete architecture

The conceptual architecture of OpenCV that was created during assignment 1 can be seen in Figure 1, and predicted the use of OpenCV as a layered architecture under minimal cross-coupling of independent subsystems. The core module was at the lowest level and offered mathematical and data-handling foundations to all higher levels [4]. The imagecodecs subsystem which handles image input and output as well as videoio were only to be dependent on core in the conceptual architecture and placed in the layer after Core [2]. The more advanced modules, like features2d and ml, were also placed above these layers, using functionality of the core and imgproc [5]. The conceptual architecture displays the different layers of OpenCV, showing that the higher modules depend only on lower ones, and optional dependencies were not included to maintain modularity. Testing and example utilities were also not part of the conceptual architecture diagram.

The concrete architecture of OpenCV can be seen in Figure 2, and was generated by reverse engineering using LSEdit, and verified with the CMakeLists.txt of each module of OpenCV. This diagram displays a more connected architecture. Although this establishes a more widely-layered and modular style, the real dependencies amongst modules are more subtle and less strictly hierarchical. It has been revealed during the analysis that there are certain divergences, such as some of the dependencies could not be found in the code, whereas some were there unexpectedly. To give an example, the conceptual model showed videoio and imgcodecs as data acquisition modules operating in parallel to each other, whereas the concrete architecture provides a clear dependence of videoio on imgcodecs to process image sequences [2]. On the other hand, the Machine Learning(ml) module, which was conceptually believed to rely on imgproc to process the image, actually solely relies on the core one, which has been confirmed in the CMake configuration file [2].

4.2. Comparison of conceptual and concrete architecture at high level (Reflexion analysis) & Modifications

The high-level conceptual architecture was derived based on the review of the OpenCV documentation and the steps in a typical computer vision pipeline described in the article “What

are the main steps in a typical Computer Vision Pipeline?" which outlines the steps for processing and analysis of visual data. These main steps include image acquisition, preprocessing, image segmentation, feature extraction,object detection,object recognition and classification,post processing, as well as visualization and interpretation [6]. This conceptual architecture is shown in figure 3 below. However, as highlighted in red in the concrete architecture shown in figure 4 (Reflexion architecture) below, there are several divergences and discrepancies between the conceptual and concrete architecture derived through analysis of code using Understand and LSEdit.

Both architecture diagrams are layered, however there are differences in the order of these layers due to the discovery of the missing dependencies. For example, the Graph-API module was moved to an upper layer as it has an optional dependency on the video module. Also the imgproc which handles image processing was moved to a lower layer as both image and video input/output modules depend on this module combining the data acquisition and image preprocessing into one layer. The label of the layer in which the photo module is placed was changed from preprocessing to "advanced image processing" to accurately reflect the functionality of the single module in this layer.

As seen in the figures below, several dependencies were missing and one incorrect dependency was found in the conceptual architecture. As described in section 4.1, contrary to the conceptual diagram, the module ml doesn't depend on the imgproc module. According to the configuration file for this model CMakeLists, this module only depends on the core module based on the following line; "ocv_define_module(ml opencv_core WRAP java objc python)". Furthermore, there were eight missing dependencies in the conceptual architecture. With the exception of the dependency of videoio module on imgcodecs and imgcodecs dependency on imgproc, the other missing dependencies were optional modular dependencies that were added due to several reasons including refactoring, performance optimization, user convenience or to maintain compatibility with older versions of OpenCV. Analysis of code commits using DeepWiki was used to explore the rationale behind the existence of these missing dependencies as described in section 4.3.

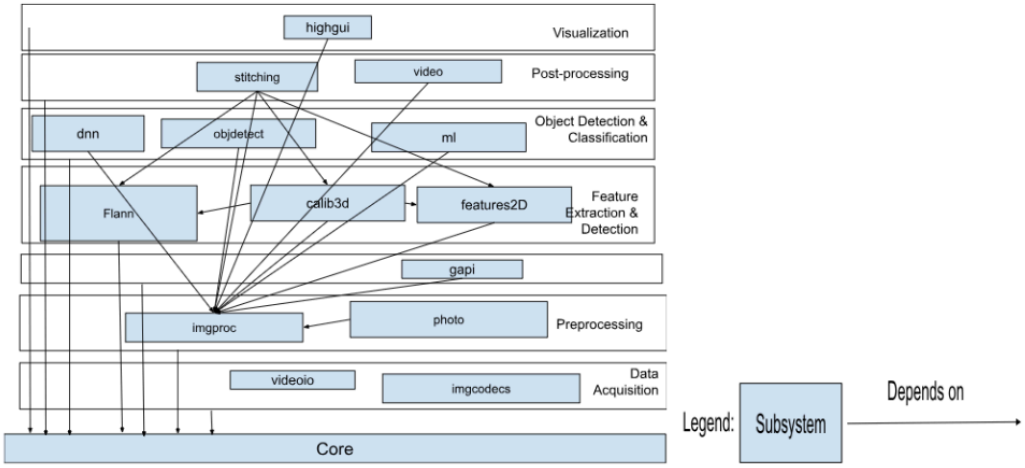

Figure 3. Conceptual Architecture

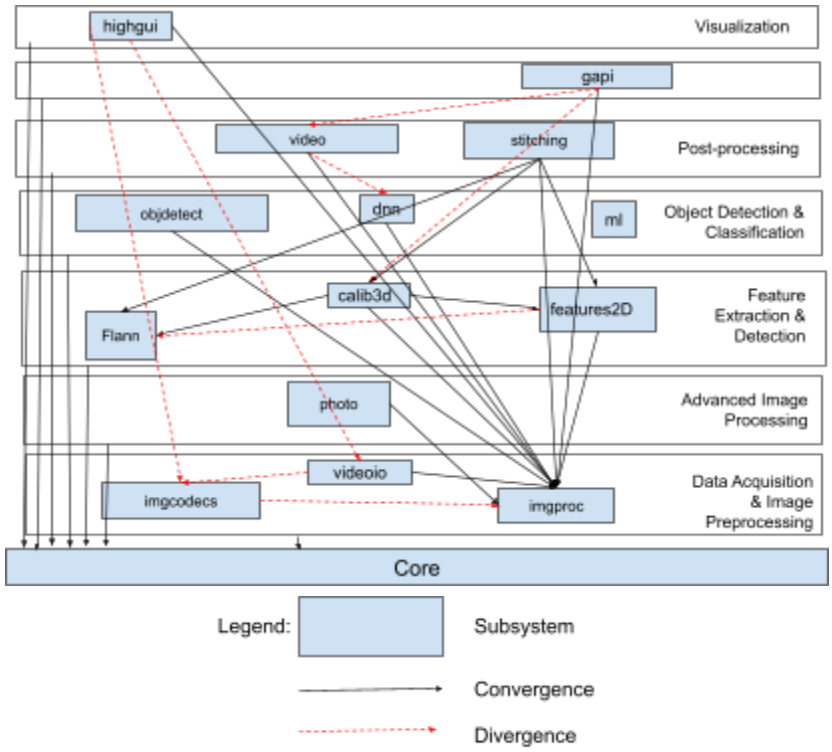

Figure 4 . Reflexion Architecture

4.3. Rationales behind Dependency Divergences in the Concrete Architecture

The concrete architecture derived through analysis of code using Understand was visualized in LSEdit and concrete examples of the modular dependencies that were missing in the conceptual architecture were analyzed. An overview of each missing dependency is given below.

Which ?	The module imgcodecs depends on imgproc. Concrete Example From LSEdit: opencv-4.12.0/modules/imgcodecs/src/grfmt_gif.cpp cLinks opencv-4.12.0/modules/imgproc/src/color.cpp
Who?	Super [7]
When?	2024-12-07, part of the initial GIF decoder implementation.
Why?	The imgcodecs module has a required dependency on the imgproc module as shown in the build configuration file CMakeLists.txt:“ocv_add_module(imgcodecs opencv_imgproc WRAP java objc python)”. The concrete implementation example of this dependency is the dependency of grfmt_git.cpp from imgcodecs on color.cpp from imgproc as color conversion functionality is used in the GIF decoder. Specifically, the GIF decoders use

	cvtColor() from imgproc to convert decoded images between different color formats [7].

Which ?	The module videoio depends on imgcodecs. Concrete Example From LSEdit: opencv-4.12.0/modules/videoio/src/cap_images.cpp cLinks imgcodecs.ss ->opencv-4.12.0/modules/imgcodecs/src/loadsave.cpp et al.
Who?	Vadim Pisarevsky [8]
When?	2010-05-11
Why?	The videoio module has a required dependency on the imgcodecs module as it needs its functionalities for reading/writing individual frames in various image formats, handling image format conversions during video encoding/decoding and frame extraction and video thumbnail generation. This dependency is listed in the configuration file : “ocv_add_module(videoio opencv_imgproc opencv_imgcodecs WRAP java objc python)”. A concrete example of this dependency is how cap_images.cpp in the videoio module relies on functionality from imgcodecs. The file implements image-sequence capture and writing, which requires reading and writing individual image files from disk. To do this, it calls imread(), implemented in loadsave.cpp within the imgcodecs module [8].

Which?	G-API module has an optional dependency on the video module.
Who?	Orest Chura [9]
When?	April 8, 2020

Why?	The dependency of G-API module on video module is optional and and it can function without the video module as show in the CMakeLists.txt:
	opencv/opencv modules/gapi/CMakeLists.txt
	`set(the_description "OpenCV G-API Core Module")`
	`ocv_add_module(gapi`
	`  REQUIRED`
	`  opencv_imgproc`
	`  OPTIONAL`
	`  opencv_video opencv_calib3d`
	`  WRAP`
	`  python`
	`)`
	This dependency was added to support video processing operations in G-API’s CPU backend[9].

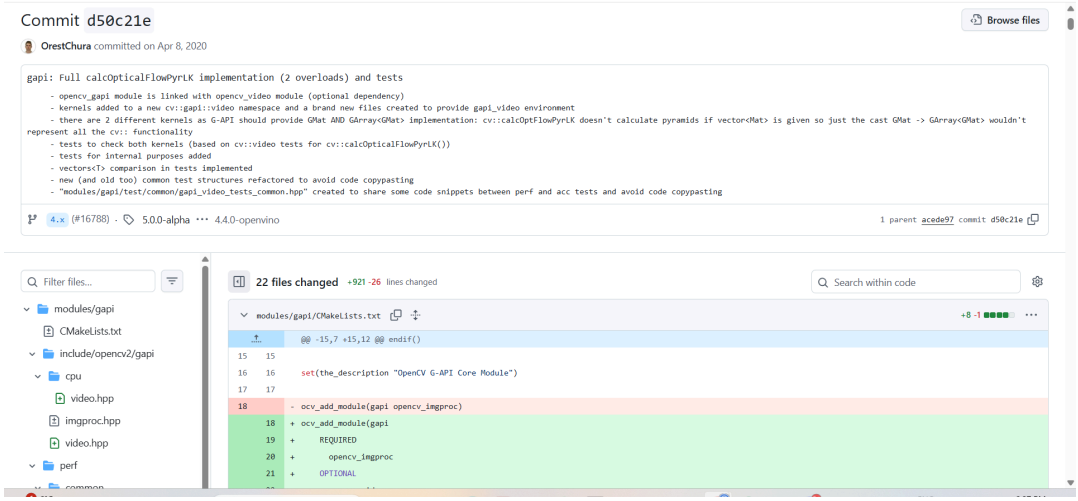

Figure 5 . Code commit that added G-API module's optional dependency on video module[8].

Which?	G-API module has an optional dependency on the video module. Concrete Example From LSEdit: opencv-4.12.0/modules/gapi/test/cpu/gapi_ocv_stateful_kernel_test.cpp cLinks video.ss-> opencv-4.12.0/modules/video/src/bgfg_gaussmix2.cpp
Who?	AsyaPronina (Anastasiya Pronina) [10]
When?	2020-06-04
Why?	The implementation example is a test dependency where the test file gapi_ocv_stateful_kernel_tests.cpp uses video module functionally(cv::BackgroundSubtractor) to test G-API's stateful kernel functionality. This stateful kernel testing framework was introduced in OpenCV 4.4 [10].

Which?	G-API module has an optional dependency on the calib3d module. Concrete Example From LSEdit: opencv4.12.0/modules/gapi/src/backednds/cpu/gcpustero.cpp cLinks calib3d.ss ->opencv-4.12.0/modules/calib3d/src/stereobm.cpp
Who?	aDanPin[11]
When?	2021-03-31
Why?	The dependency of G-API module on calib3d module is optional and and it can function without the calib3d module as show in the CMakeLists.txt:

	opencv/opencv modules/gapi/CMakeLists.txt
	`set(the_description "OpenCV G-API Core Module")`
	`ocv_add_module(gapi`
	`  REQUIRED`
	`  opencv_imgproc`
	`  OPTIONAL`
	`  opencv_video opencv_calib3d`
	`  WRAP`
	`  python`
	`)`
	The concrete example is the dependency of the gcpustero.cpp in G-API module on stereo matching algorithms like StereoBM implemented in the calib3d module. Through this dependency, G-API exposes stereo-vision functionality by wrapping calib3d's algorithms in its graph-based API. When the calib3d module is present, G-API provides stereo-vision features; if it's absent, those capabilities are excluded from the build [11].

Which?	Video module has an optional dependency on the dnn module. Concrete Example From LSEdit: opencv-4.12.0/modules/video/src/tracking/tracker_vit.cpp cLinks dnn.ss->opencv-4.12.0 /modules/dnn/src/net.cpp
Who?	lpylpy0514 (PengyuLiu)[12]
When?	2023-09-19
Why?	The dependency of Video module on dnn module is optional and and it can function without the dnn module as show in the CMakeLists.txt: "ocv_add_module(video REQUIRED opencv_core opencv_imgproc OPTIONAL opencv_dnn WRAP python java)" The video module uses dnn for deep learning-based tracking algorithms. The concrete example is the dependency of the tracker_vit.cpp in Video module on dnn functionality (dnn:: Net class implemented in net.cpp) to load and run vision transformer neural network model for object tracking [12].

Which?	The module features2d has an optional dependency on the flann module. Concrete Example From LSEdit: opencv-4.12.0/modules/features2d/src/matchers.cpp cLinks flann.ss->opencv-4.12.0 /modules/flann/src/miniflann.cpp
Who?	Multiple Contributions: -Vadim Pisarevsky: foundational FLANN wrapper code in miniflann.cpp [13] -Maria Dimashova: created the FlannBasedMatcher class in matchers.cpp [5]
When?	2011-07-13 (Vadim Pisarevsky) [13]

	2010-10-29(Maria Dimashova) [5]
Why?	The dependency of features2d module on flann module is optional and and it can function without the video module as show in the CMakeLists.txt "ocv_define_module(features2d opencv_imgproc ${debug_modules} OPTIONAL opencv_flann WRAP java objc python js)". The concrete implementation example of this dependency is the dependency of the FlannBasedMatcher class in matchers.cpp in features2d module on flann::Index class implemented in miniflann.cpp in flann module for fast approximate nearest neighbor search when matching feature descriptors [5],[13].

Which?	The module highgui has an optional dependency on imgcodecs. Concrete Example From LSEdit: opencv-4.12.0/modules/highgui/include/opencv2/highgui.hpp cLinks imgcodecs.ss ->opencv-4.12.0/modules/imgcodecs/include/opencv2/imgcodecs.hpp
Who?	Alexander Alekhin [14]
When?	2016-08-25

Why?	The dependency of highgui module on imgcodecs module is optional and and it can function without the imgcodecs module as show in the CMakeLists.txt:
	opencv/opencv modules/highgui/CMakeLists.txt
	`set(the_description "High-level GUI")`
	
	`if(ANDROID)`
	`  ocv_add_module(highgui opencv_imgproc OPTIONAL opencv_imgcodecs opencv_videoio WRAP python)`
	`else()`
	`  ocv_add_module(highgui opencv_imgproc OPTIONAL opencv_imgcodecs opencv_videoio WRAP python java)`
	`endif()`
	This change was part of OpenCV 3.0 modularization effort; the original highgui module was split into three separate modules including imgcodecs, videoio, and highgui. This dependency was added to maintain backward compatibility from OpenCV 2.x where image I/O functions were part of the original highgui module.This dependency is also included to provide convenient access to image I/O functions (imread, imwrite) when users include the highgui header [14],[15].

Which?	The module highgui has an optional dependency on videoio. Concrete Example From LSEdit: opencv-4.12.0/modules/highgui/include/opencv2/highgui.hpp cLinks opencv-4.12.0/modules/videoio/include/opencv2/videoio.hpp
Who?	Alexander Alekhin [14]
When?	2016-08-25
Why?	The dependency of highgui module on videoio module is optional and and it can

function without the videoio module as show in the CMakeLists.txt:

opencv/opencv modules/highgui/CMakeLists.txt

```
set(the_description "High-level GUI")

if(ANDROID)
  ocv_add_module(highgui opencv_imgproc OPTIONAL opencv_imgcodecs opencv_videoio WRAP python)
else()
  ocv_add_module(highgui opencv_imgproc OPTIONAL opencv_imgcodecs opencv_videoio WRAP python java)
endif()
```

The highgui header conditionally includes videoio.hpp to provide convenient access to video I/O functions like VideoCapture and VideoWriter when highgui is used. This maintains backward compatibility from OpenCV 2.x as these functions were part of the original highgui module[14],[15].

5. Subsystem-Level Detailed Analysis-Image Processing Subsystem

5.1. Descriptions of the conceptual and the concrete architecture of imgproc

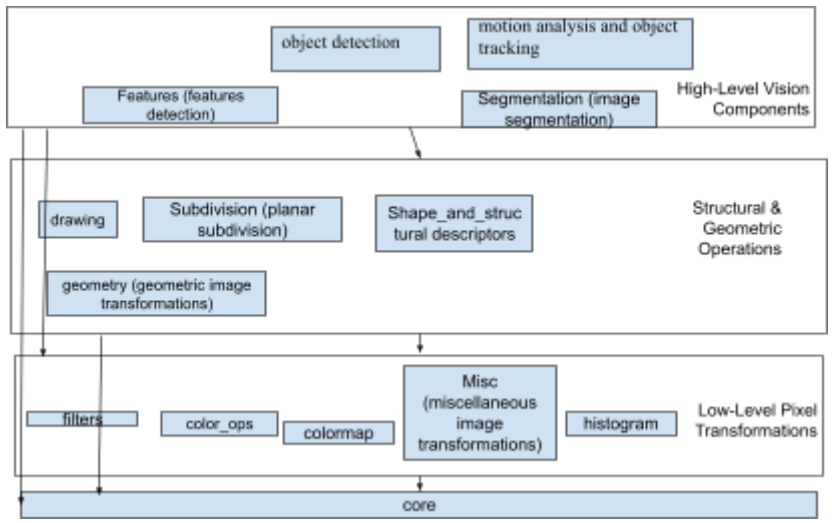

Figure 6. Conceptual architecture of imgproc

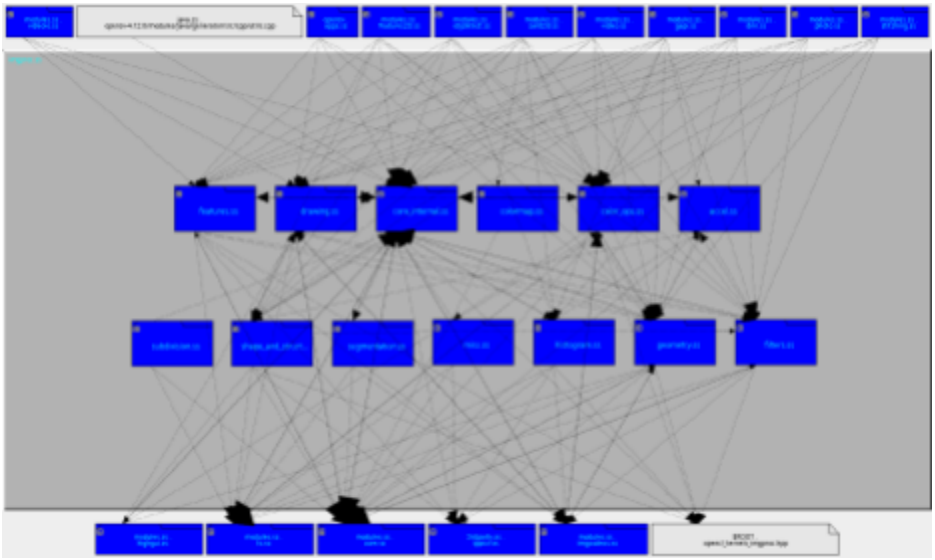

Figure 7. Concrete Architecture of “imgproc” module captured from LSEdit.

For better resolution, please view this link: imgproc.png

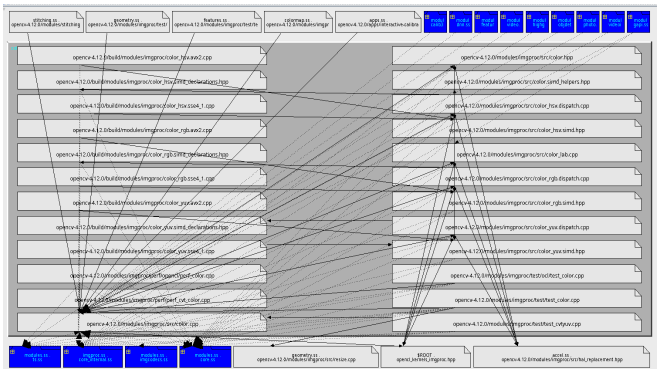

Figure 8. Concrete Architecture of “color_ops” which represents the colour operations subsystem of “imgproc” captured from LSEdit.

For better resolution, please view this link: colour_op.png

The imgproc module serves as the core Image Processing subsystem in OpenCV. According to OpenCV’s documentation, “imgproc” modules contains image processing functions that enable tasks such as image filtering, geometric image transformations, miscellaneous image transformation, drawing functions, color space conversions, colour maps, planar subdivision, histograms, structural analysis and shape descriptors, motion analysis and object tracking, feature detection, object detection, and image segmentation [2].The conceptual architecture of imgproc derived from this documentation presents it as a structured hierarchy composed of three broad functional layers: low-level pixel transformations, structural and geometric operations, and high-level vision components. At the lowest level, imgproc includes operations that directly manipulate pixels, such as image filtering, thresholding, color space conversions, color mapping, and histogram computation or equalization[2]. The second layer

consists of structural and geometric operations that deal with multi-pixel relationships or image geometry, including functions for resizing, warping, rotating, detecting contours, computing connected components, and performing planar subdivision [2]. At the highest level, imgproc offers feature detection, basic object detection, and segmentation algorithms such as watershed and grabCut [2]. In the conceptual model, each of these layers is clearly delineated, with data flowing upward from low-level pixel processes to more complex structural and semantic operations. Conceptually, imgproc depends on the core module for data structures and matrix operations, while higher-level modules such as imgcodecs, video, and highgui simply consume imgproc functionality. This representation emphasizes a clean and modular architecture with minimal cross-layer dependencies.

The concrete implementation of imgproc reveals a more complex and interconnected subsystem than the conceptual architecture suggests. Rather than three neatly separated layers, the actual imgproc implementation spans numerous subsystems such as segmentation.ss, morphology.ss, geometry.ss, filters.ss, features.ss, core_internal.ss, and color_ops.ss, many of which interact in ways not captured in the conceptual model. Subsystems like core_internal.ss and filters.ss function as central utility hubs because they provide widely used low-level routines while simultaneously depending on other modules for specialized functionality. This creates bidirectional dependencies rather than the one-way, hierarchical flow implied by the conceptual diagram. Furthermore, the concrete implementation introduces hardware-specific optimizations through SIMD, AVX, SSE, and sometimes CUDA acceleration, resulting in additional files and complexity not represented in the conceptual architecture.

A second-level subsystem that highlights these discrepancies particularly well is color_ops. Conceptually, color_ops is a compact, self-contained functional unit within the low-level pixel transformation layer, responsible for converting between color spaces such as RGB, HSV, Lab, and YUV. In this idealization, the module has a well-defined interface and is treated as a foundational component that other subsystems can depend on when a particular color representation is required. The conceptual view presents color_ops as a simple block that supports later stages of the pipeline, including geometric transformations and segmentation algorithms, without showing internal complexity or platform-specific considerations.

However, the concrete implementation of color_ops is distributed across numerous specialized files, including color_rgb.avx2.cpp, color_lab.cpp, color_yuv.cpp, color_hsv.sse4.cpp, and various SIMD helper files, each targeting different hardware instruction sets or algorithmic variants. Rather than operating in isolation, color_ops interacts extensively with core_internal.ss for memory access and pixel manipulation, with accel.ss for hardware dispatching, and with geometry.ss and filters.ss when color information is required during geometric warping or filtering tasks. As a result, color_ops exhibits both high fan-in, because many parts of imgproc rely on color conversions, and high fan-out, because it depends on multiple utility and acceleration modules to execute its tasks efficiently.

5.2. Comparison of conceptual and concrete architecture of imgproc

The conceptual architecture for imgproc, as outlined earlier in the report, presents the subsystem as three functional layers: low-level pixel operations, structural/geometric operations, and higher level segmentation and feature extraction tasks. The conceptual view assumes that

each functional group is internally cohesive, minimally dependent on others, and primarily dependent only on the core.

The concrete architecture recovered through LSEdit and Understand shows a substantially different organization. A tightly interconnected set of subsystems as previously mentioned such as ,filters.ss and geometry.ss, with many cross-module dependencies that contradict the conceptual separation.

Both “object detection” and “motion analysis/object tracking” appear as categories under imgproc in the OpenCV documentation, but they do not contain any real implementations inside the imgproc module [16]. Since no source files were found for these two areas, they were not shown in the final LSEdit diagram. Although the tags @defgroup imgproc_object and @defgroup imgproc_motion appear in the imgproc header file, they seem to function only as documentation placeholders rather than actual code modules [16]. The actual code for these operations lives in regular .cpp files inside opencv/modules/imgproc/src/, such as templmatch.cpp for template matching [17] and accum.cpp or phasecorr.cpp for motion-analysis-related functions [18]. Since these .cpp files are physically part of imgproc’s general source directory rather than a dedicated “object” or “motion” folder, the containment file obtained from the concrete architecture groups them into broader subsystems like filters, geometry, or misc instead of creating separate nodes.

5.3. Rationales behind Dependency Divergences in the Concrete Architecture of Imgproc

The differences between the conceptual and concrete architectures of the imgproc subsystem come from practical implementation constraints, performance requirements and hardware considerations. The rationales below explain why these discrepancies exist and how we found them through our analysis of the imgproc subsystem.

One major reason for the differences between the conceptual and concrete architecture is OpenCV’s strict performance requirements. Conceptually, imgproc was modeled as a set of clean isolated functional layers (pixel operations → structural/geometric operations → segmentation and features) (Section 5.1). However, our concrete analysis showed extensive cross-module dependencies. This is because low-level operations such as convolution, image smoothing, interpolation and color conversions share optimized routines instead of being fully separated (Figure 4). This was determined by inspecting LSEdit dependency graphs, which showed many connections between the filtering utilities.

Another reason for the discrepancies between the conceptual and concrete architecture is because of the mathematical and computational foundations required by nearly all the image processing routines. Like stated earlier, the conceptual imgproc suggests that pixel operations, structural/geometric analysis and segmentation/features should all form distinct layers. However, functions like resize, warp, contour analysis, and shape descriptors all call into shared interpolation, filtering, and matrix utilities (Figure 4) This tight coupling exists because many algorithms depend on the same underlying routines for interpolation, gradient computation, and matrix manipulation. We obtained this rationale by tracing dependencies in LSEdit, which showed that subsystems like geometry.ss depend on filters.ss (Figure 4).

A third rationale behind the differences is the influence of hardware acceleration on OpenCV's implementation. Conceptually, higher-level tasks such as segmentation and feature detection were treated as distinct higher-layer operations (Section 5.1). However, the concrete implementation of these functionalities across multiple hardware-specific files shows dependencies between subsystems that were not present conceptually (Section 5.1). This occurs because OpenCV optimizes critical operations across CPU architectures, forcing subsystems to share low-level implementations instead of remaining isolated [19]. As a result, multiple imgproc subsystems converge on the same SIMD kernels which can be seen in the concrete architecture.

6. Use Cases

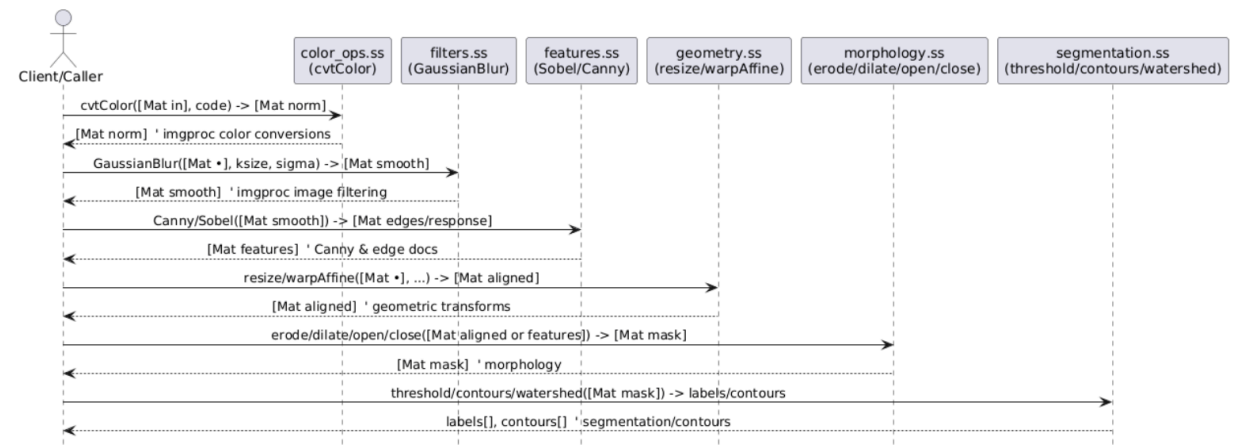

Figure 9. Sequence Case from Assignment created using [20]

This sequence diagram from Assignment 2 remains unchanged from Assignment 2 because it already captures all major imgproc subsystems making it representative of how the user could utilize these subsystems through function calls [21-27].

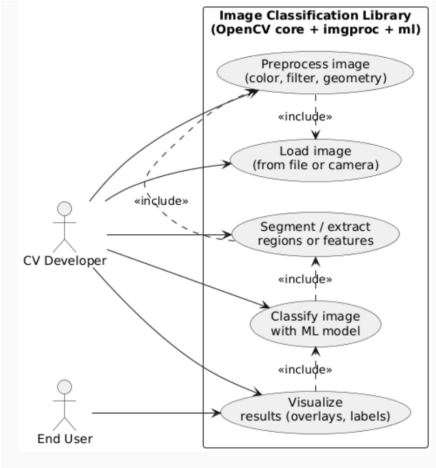

Figure 10. Use Case Image Classification created using [20]

The use case diagram above shows how a CV developer and an end user interact with an image classification pipeline built using OpenCV's core, imgproc, and ml modules. The library performs several steps including preprocessing the image, loading input, segmenting or

extracting features, classifying with an ML model, and visualizing results [28]. Each stage represents a typical part of the computer vision workflow, with the developer invoking these operations to produce outputs the end user can view. The diagram highlights how these functions combine to form a complete image-classification process.

This state diagram outlines the typical flow of an image through an OpenCV-based image classification pipeline. The process starts by loading an image, which is then converted to grayscale and optionally blurred or normalized as part of preprocessing [22][23][28]. After resizing to a standard resolution [25], the workflow performs segmentation or region extraction, such as thresholding or contour based selection [27]. The segmented image is then used for feature extraction and machine-learning-based classification, consistent with the OpenCV and online tutorials [24][28]. Finally, the resulting prediction may be visualized on the image using OpenCV drawing utilities [26], while invalid inputs or processing failures transition to an error state.

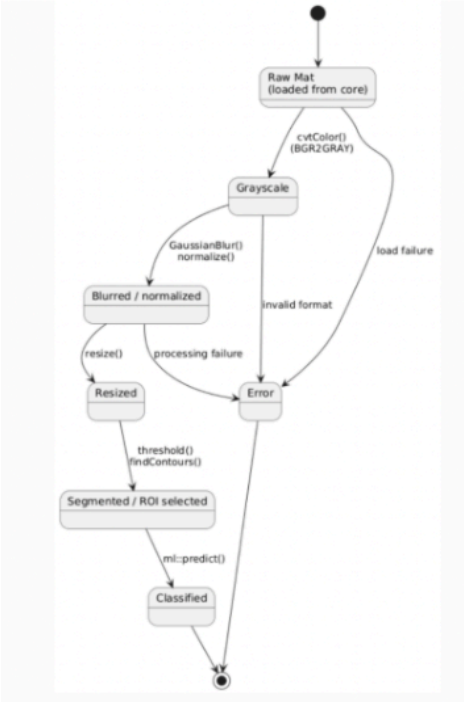

Figure 11. State Diagram created using [20].

7. Lessons Learned

There are differences between OpenCV's conceptual and concrete architecture which are highlighted by the reflexion analysis. Unpredicted dependencies and hidden subsystem interactions were discovered using Understand and LSEdit. Intricate subsystem analysis emphasized coupling that was deeper than what was predicted. On the other hand, architectural divergences were revealed through advanced-level comparisons. Motive revolving around design and the comprehension of module relationships were both improved by visual dependency graphs. The cruciality of iterative validation was highlighted by the discoveries to notify developments in the future and to ensure architectural consistency.

8. Conclusion

In conclusion, this discrepancy analysis reveals that the conceptual architecture of OpenCV presents a clean and layered modular structure while the concrete architecture reflects a more coupled performance based system. At the top level, subsystems diverge from the intended design due to optional dependencies, backward-compatibility constraints and architectural drift introduced by years of incremental contributions. At the subsystem level, specifically for imgproc, there are also many differences because of shared computation routines and hardware-accelerated implementations resulting in tightly coupled structures that contrast with the idealized conceptual layering. By analyzing these discrepancies using LSEdit and Understand, we were able to uncover the reasons behind these deviations and identified how practical constraints shape OpenCV's actual architecture. These insights clarify why the divergences exist and highlight how these differences naturally emerge in a large open source system like OpenCV.

9. References

[1] "OpenCV: OpenCV modules," *Opencv.org*, 2025. https://docs.opencv.org/4.12.0/

[2] "OpenCV: Image Processing." Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d7/dbd/group__imgproc.html

[3] "opencv/opencv," DeepWiki. Accessed: Nov. 21, 2025. [Online]. Available: https://deepwiki.com/opencv/opencv

[4] "Core functionality (core module)," OpenCV Documentation, accessed Oct. 28, 2025. [Online]. Available: https://docs.opencv.org/4.x/d0/de1/group__core.html

[5] "modified features2d interface; added algorithmic test for DescriptorM… · opencv/opencv@69e329c · GitHub." Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/69e329c9

[6] "What are the main steps in a typical Computer Vision Pipeline?," GeeksforGeeks. Accessed: Oct. 10, 2025. [Online]. Available: https://www.geeksforgeeks.org/computer-vision/what-are-the-main-steps-in-a-typical-computer-vision-pipeline/

[7] opencv, "Merge pull request #25691 from redhecker:gifSupport · opencv/opencv@082cd7a," GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/082cd7a74eab9a435c414e4fe7a5c0803d4d3b4d

[8] opencv, "'atomic bomb' commit. Reorganized OpenCV directory structure · opencv/opencv@127d664," GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/127d6649a1c83397bf42799ac807af41aa507b30

[9] "gapi: Full calcOpticalFlowPyrLK implementation (2 overloads) and tests · opencv/opencv@d50c21e · GitHub." Accessed: Nov. 16, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/d50c21e5

[10] opencv, "Enable stateful kernels in G-API OCV Backend · opencv/opencv@b083c20," GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/b083c20eb2eba7227f66dd10837609db9704fec6

[11] opencv, "Adding stereo operation and tests · opencv/opencv@e6f52b0," GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/e6f52b0996d8e4f96ed13e501b4166805f09024b

[12] opencv, "Merge pull request #24201 from lpylpy0514:4.x · opencv/opencv@70d7e83," GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/70d7e83dca7824e7e9d09e03630d1134628dc70a

[13] opencv, "upgraded to FLANN 1.6. Added miniflann interface, which is now used i… · opencv/opencv@562914e," GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/562914e33bea921a0a235fbf2b6bba6830644e16

[14] opencv, "fix modules deps · opencv/opencv@063ac74," GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/063ac74369199c1cfcaefe266d963ea1c21db75d

[15] "OpenCV: Transition guide." Accessed: Nov. 21, 2025. [Online]. Available: https://docs.opencv.org/4.x/db/dfa/tutorial_transition_guide.html

[16] "imgproc.hpp source code [opencv/modules/imgproc/include/opencv2/imgproc.hpp] - Codebrowser," Codebrowser.dev, 2019.

https://codebrowser.dev/opencv/opencv/modules/imgproc/include/opencv2/imgproc.hpp.html (accessed Nov. 21, 2025).
[17] Github "OpenCV templmatch.cpp," Github.com, 2025. https://github.com/opencv/opencv/blob/4.x/modules/imgproc/src/templmatch.cpp (accessed Nov. 21, 2025).
[18] opencv, "opencv/modules/imgproc/src at 4.x · opencv/opencv," GitHub, 2025. https://github.com/opencv/opencv/tree/4.x/modules/imgproc/src (accessed Nov. 21, 2025).
[19] "Hardware Abstraction Layer (HAL) – SIMD Intrinsics," OpenCV Documentation, accessed Nov. 21, 2025. [Online]. Available: https://docs.opencv.org/4.x/df/d91/group__core__hal__intrin.html
[20] "PlantUML," PlantUML.com, https://plantuml.com/ (accessed Oct. 28, 2025).
[21] "CV::Mat class reference," OpenCV, https://docs.opencv.org/3.4/d3/d63/classcv_1_1Mat.html (accessed Oct. 31, 2025).
[22] "Color conversions," OpenCV, https://docs.opencv.org/3.4/de/d25/imgproc_color_conversions.html (accessed Oct. 31, 2025).
[23] "Smoothing Images," OpenCV, https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html (accessed Oct. 31, 2025).
[24] "Feature detection," OpenCV, https://docs.opencv.org/4.x/dd/d1a/group__imgproc__feature.html (accessed Oct. 31, 2025).

[25] "OpenCV: Geometric Image Transformations," docs.opencv.org. https://docs.opencv.org/3.4/da/d54/group__imgproc__transform.html (accessed Oct. 31, 2025).

[26] "OpenCV: Drawing Functions." Accessed: Oct. 31, 2025. [Online]. Available: https://docs.opencv.org/4.x/d6/d6e/group__imgproc__draw.html

[27] "Morphological Transformations," OpenCV, https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html (accessed Oct. 31, 2025).
[28] "Feature extraction and image classification using OpenCV - GeeksforGeeks." Accessed: Oct. 10, 2025. [Online]. Available: https://www.geeksforgeeks.org/computer-vision/feature-extraction-and-image-classification-using-opencv/